

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ХАРКІВСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ РАДІОЕЛЕКТРОНІКИ

Кафедра «Програмна інженерія»

ЗВІТ

з лабораторної роботи №4

«Рефакторинг коду та проведення Code Review

за індустріальними стандартами»

з дисципліни **«Основи програмної інженерії»**

Виконав:

ст. гр. ПЗПІ-25-5

Козирь В.С.

Прийняв:

Гребенюк В. О

Харків 2026

4 РЕФАКТОРИНГ КОДУ ТА ПРОВЕДЕННЯ CODE REVIEW ЗА ІНДУСТРІАЛЬНИМИ СТАНДАРТАМИ

4.1 Мета роботи

Набуття практичних навичок із проведення Code Review за індустріальними стандартами з використанням GitHub Pull Requests. Оволодіння методами ідентифікації та класифікації типових проблем якості коду (code smells). Отримання досвіду застосування рефакторинг-операцій із верифікацією збереження поведінки через регресійне тестування.

Хід роботи

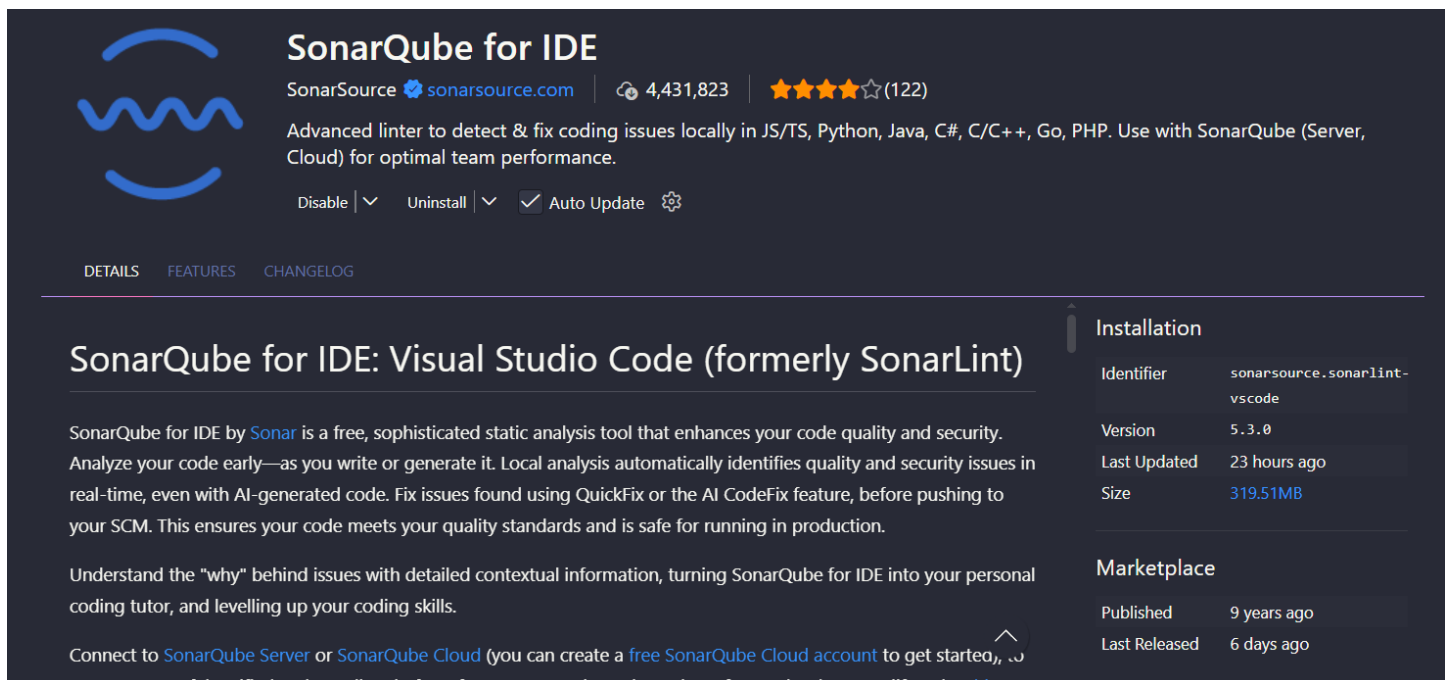
1. Переконавшись, що код модуля та тести з ЛР 03 знаходяться у GitHub-репозиторії.

Посилання на виконану ЛР3:

<https://github.com/Mykhailo-Berkuta/2Win/tree/feature/Kozyr/lab%203>

2. Встановити SonarQube for IDE (SonarLint) у VS Code. Запустити аналіз свого коду та зафіксувати початкову кількість issues (скріншот «до»).

Встановлений SonarQube for IDE



The screenshot shows the Visual Studio Code marketplace page for the SonarQube for IDE extension. The header includes the SonarSource logo, the extension name 'SonarQube for IDE', the publisher 'SonarSource', the website 'sonarsource.com', a download count of 4,431,823, and a star rating of 4.5 stars (122 reviews). Below this, a description states it is an 'Advanced linter to detect & fix coding issues locally in JS/TS, Python, Java, C#, C/C++, Go, PHP. Use with SonarQube (Server, Cloud) for optimal team performance.' There are controls for 'Disable', 'Uninstall', and 'Auto Update' (which is checked). Navigation tabs for 'DETAILS', 'FEATURES', and 'CHANGELOG' are present. The main content area is titled 'SonarQube for IDE: Visual Studio Code (formerly SonarLint)' and describes the tool as a free, sophisticated static analysis tool that enhances code quality and security. It mentions real-time analysis, AI-generated code support, and the ability to fix issues using QuickFix or AI CodeFix. A section titled 'Installation' on the right lists the identifier 'sonarsource.sonarlint-vscode', version '5.3.0', last updated '23 hours ago', and size '319.51MB'. A 'Marketplace' section on the right shows it was published '9 years ago' and last released '6 days ago'.

SonarQube for IDE
SonarSource | sonarsource.com | 4,431,823 | ★★★★★ (122)
Advanced linter to detect & fix coding issues locally in JS/TS, Python, Java, C#, C/C++, Go, PHP. Use with SonarQube (Server, Cloud) for optimal team performance.
Disable | Uninstall | ☒ Auto Update

DETAILS | FEATURES | CHANGELOG

SonarQube for IDE: Visual Studio Code (formerly SonarLint)

SonarQube for IDE by Sonar is a free, sophisticated static analysis tool that enhances your code quality and security. Analyze your code early—as you write or generate it. Local analysis automatically identifies quality and security issues in real-time, even with AI-generated code. Fix issues found using QuickFix or the AI CodeFix feature, before pushing to your SCM. This ensures your code meets your quality standards and is safe for running in production.

Understand the "why" behind issues with detailed contextual information, turning SonarQube for IDE into your personal coding tutor, and levelling up your coding skills.

Connect to SonarQube Server or SonarQube Cloud (you can create a free SonarQube Cloud account to get started, or create a powerful, unified code quality platform for your team throughout the software development lifecycle. This

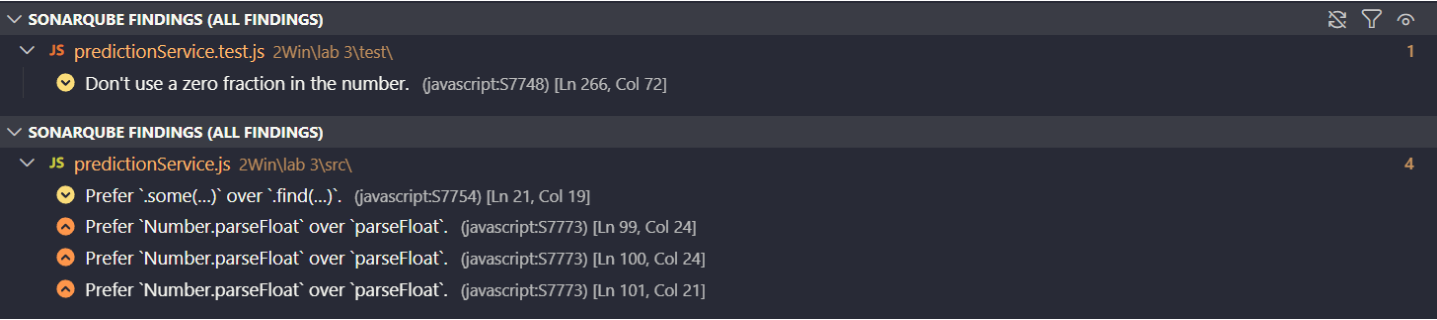
Installation

Identifier	sonarsource.sonarlint-vscode
Version	5.3.0
Last Updated	23 hours ago
Size	319.51MB

Marketplace

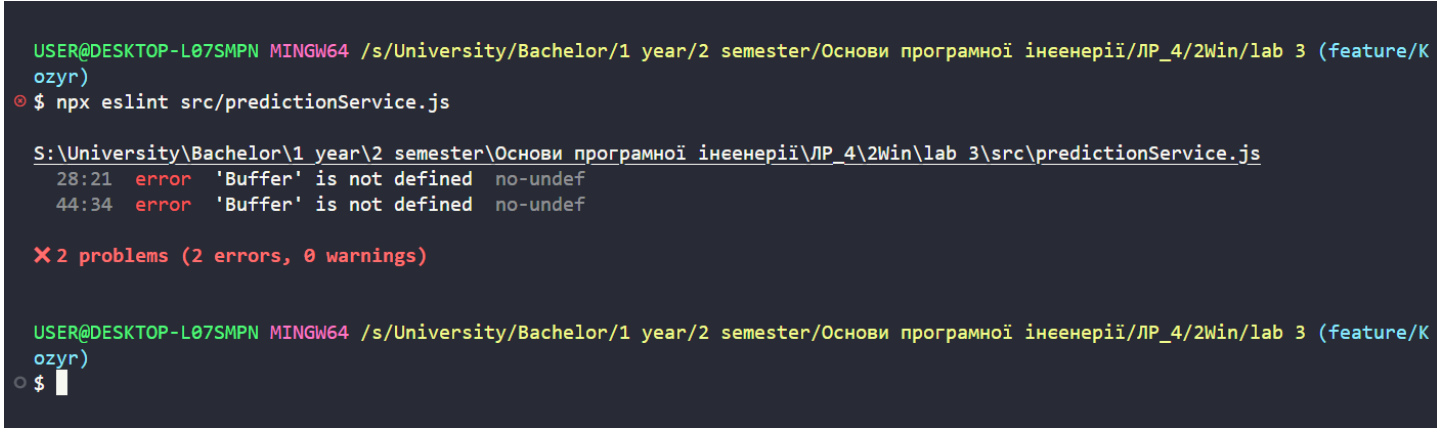
Published	9 years ago
Last Released	6 days ago

Аналіз свого коду кількості issues



3. Запустити лінтер (Ruff для Python: ruff check src/; ESLint для JS/TS; Checkstyle для Java). Зафіксувати кількість попереджень.

Запуск Eslint



4. Отримати доступ до репозиторію одногрупника (fork або запрошення як collaborator).

Репозиторій одногрупника: <https://github.com/Mykhailo-Berkuta/2Win/tree/feature/Berkuta>

5. Створити нову гілку (review-[ваше-ім'я]) та відкрити Pull Request.

Моя гілка: <https://github.com/Mykhailo-Berkuta/2Win/tree/review-Kozyr>

Мій Pull Request: <https://github.com/Mykhailo-Berkuta/2Win/pull/8>

6. Провести Code Review: ретельно проаналізувати код одногрупника та виявити щонайменше 5 проблем якості коду.

№	Рядок коду	Категорія Code Smell	Рекомендація
1	const path = require('path');	Мертвий код (Dead Code)	Видалити невикористаний імпорт - він збільшує когнітивне навантаження без будь-якої користі

№	Рядок коду	Категорія Code Smell	Рекомендація
2	filterByStatus() - this._store.get(userId) ?? [] дублює логіку getBetsByUser(), при цьому валідація userId пропущена	DRY	Замінити на виклик this.getBetsByUser(userId), що усуне і дублювання, і пропущену валідацію
3	const fs = require('fs');	Dead Code	Видалити невикористаний імпорт
4	const total = this.favoriteCategories.length; в addCategory()	Dead Code	Видалити - змінна обчислюється, але ніколи не читається
5	removeCategory() повертає boolean, але жоден виклик його не використовує	Command- Query Separation	Або прибрати return (чистий command-метод), або явно задокументувати контракт і використовувати повернене значення в тестах

7. Кожну проблему задокументувати як inline-коментар у PR або GitHub Issue із зазначенням: (а) рядок коду; (б) категорія code smell; (в) конкретна рекомендація щодо виправлення.

Pull request: <https://github.com/Mykhailo-Berkuta/2Win/pull/8>

8. Отримати результати Code Review від одногрупника.

Code review від одногрупника: <https://github.com/Mykhailo-Berkuta/2Win/pull/9>

9. Виконати щонайменше 3 рефакторинг-операції за результатами отриманого review. Для кожної операції задокументувати: «БУЛО → СТАЛО → ЧОМУ» (обґрунтування).

1.

Commit 92702bb

 vsevolodkozyr committed 1 minute ago · ✓ 1 / 1

ref: винесено мінімальну довжину пароля у константу

БУЛО: `pwd.length < 6`

СТАЛО: `const MIN_PASSWORD_LENGTH = 6; → pwd.length < MIN_PASSWORD_LENGTH`

ЧОМУ: літерал 6 – Magic Number (S109). Константа самодокументує призначення, полегшує зміну в одному місці, відповідає принципу DRY.

 feature/Kozyr

1 file changed

```
-----|-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----|
All files |    100   |    100   |    100   |    100   |
predictionService.js |    100   |    100   |    100   |    100   |
-----|-----|-----|-----|-----|

Test Suites: 1 passed, 1 total
Tests:       27 passed, 27 total
Snapshots:   0 total
Time:        1.098 s
Ran all test suites.

USER@DESKTOP-L07SMPN MINGW64 /s/University/Bachelor/1 year/2 semester/Основи програмної інженерії/ЛР_4/2Win/lab 3 (feature/Kozyr)
$
```

2.

Commit 1f70545

 vsevolodkozyr committed 1 minute ago · 1/1

ref: спрощено метод validateAge – видалено надлишкову перевірку

БУЛО: if (confirmed === null || confirmed === undefined) return false; return confirmed === true;
СТАЛО: return confirmed === true;
ЧОМУ: перевірка null та undefined є Dead Code (S1854) – вирази null === true і undefined === true вже повертають false. Видалення 2 рядків без жодної зміни поведінки; TC-12, TC-13, TC-14 залишаються зеленими.

 feature/Kozyr

1 file changed

```
-----|-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----|
All files |    100   |    100   |    100   |    100   |
predictionService.js |    100   |    100   |    100   |    100   |
-----|-----|-----|-----|-----|

Test Suites: 1 passed, 1 total
Tests:       27 passed, 27 total
Snapshots:   0 total
Time:        0.61 s, estimated 1 s
Ran all test suites.

USER@DESKTOP-L07SMPN MINGW64 /s/University/Bachelor/1 year/2 semester/Основи програмної інженерії/ЛР_4/2Win/lab 3 (feature/Kozyr)
$
```

3.

Commit 2b47a8a

 vsevolodkozyr committed 1 minute ago

ref: винесено дефолтний рейтинг та вагу нічиї у константи

БУЛО: match.homeRating ?? 50, home + away + 20
СТАЛО: const DEFAULT_RATING = 50; const DRAW_WEIGHT = 20; → match.homeRating ?? DEFAULT_RATING, home + away + DRAW_WEIGHT
ЧОМУ: літери 50 і 20 – Magic Numbers (S109). Константи пояснюють роль чисел у моделі, коментар // +20 for draw weight стає зайвим.

 feature/Kozyr

1 file changed

```
-----|-----|-----|-----|-----|-----|
File      | % Stmts | % Branch | % Funcs | % Lines | Uncovered Line #s
-----|-----|-----|-----|-----|-----|
All files |    100  |    100   |    100   |    100   |
predictionService.js |    100  |    100   |    100   |    100   |
-----|-----|-----|-----|-----|
Test Suites: 1 passed, 1 total
Tests:       27 passed, 27 total
Snapshots:   0 total
Time:        1.006 s
Ran all test suites.

USER@DESKTOP-L07SMPN MINGW64 /s/University/Bachelor/1 year/2 semester/Основи програми
ozyr)
$
```

11. Повторно запустити SonarLint та лінтер. Зафіксувати кількість issues «після» (скріншот).

```
▼ SONARQUBE FINDINGS (ALL FINDINGS)
▼ JS predictionService.js lab 3\src\
  ✓ Prefer `some(...)` over `find(...)`. (javascript:S7754) [Ln 24, Col 19]
  ✗ Prefer `Number.parseFloat` over `parseFloat`. (javascript:S7773) [Ln 107, Col 24]
  ✗ Prefer `Number.parseFloat` over `parseFloat`. (javascript:S7773) [Ln 108, Col 24]
  ✗ Prefer `Number.parseFloat` over `parseFloat`. (javascript:S7773) [Ln 109, Col 21]
▼ JS predictionService.test.js lab 3\test\
  ✓ Don't use a zero fraction in the number. (javascript:S7748) [Ln 266, Col 72]

USER@DESKTOP-L07SMPN MINGW64 /s/University/Bachelor/1 year/2 semester/Основи програмної інженерії/ЛР_4/2Win/lab 3 (feature/K
ozyr)
$ npx eslint src/

S:\University\Bachelor\1 year\2 semester\Основи програмної інженерії\ЛР_4\2Win\lab 3\src\predictionService.js
  31:21  error  'Buffer' is not defined  no-undef
  47:34  error  'Buffer' is not defined  no-undef

✖ 2 problems (2 errors, 0 warnings)
```

12. Запустити обчислення метрик складності (radon або lizard). Порівняти з початковими значеннями.

До:

```
PS S:\University\Bachelor\1 year\2 semester\Основи програмної інженерії\ЛР_4\2Win\lab 3> py -m lizard .\src\
=====
NLOC    CCN    token  PARAM  length  location
-----
      3      1     11      0       3 constructor@2-4@.\src\predictionService.js
      1      1      6      0       1 (anonymous)@21-21@.\src\predictionService.js
     23      9    149      3     24 register@11-34@.\src\predictionService.js
      1      1      6      0       1 (anonymous)@42-42@.\src\predictionService.js
      6      4     56      2       6 authenticate@40-45@.\src\predictionService.js
      4      3     23      1       4 validateAge@51-54@.\src\predictionService.js
      7      1     45      5       7 constructor@58-64@.\src\predictionService.js
      3      1     17      0       3 getSummary@66-68@.\src\predictionService.js
      3      1     13      1       3 constructor@72-74@.\src\predictionService.js
     32     16    222      1     37 calculateProbability@83-119@.\src\predictionService.js
      6      1     29      0       6 (anonymous)@132-137@.\src\predictionService.js
      7      4     51      1     13 fetchOdds@126-138@.\src\predictionService.js
1 file analyzed.
=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     100         8.0      3.6      52.3         12  .\src\predictionService.js

=====
!!!! Warnings (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100) !!!!
=====
NLOC    CCN    token  PARAM  length  location
-----
     32     16    222      1     37 calculateProbability@83-119@.\src\predictionService.js
=====
Total nloc    Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
     100         8.0      3.6      52.3      12          1    0.08  0.33
PS S:\University\Bachelor\1 year\2 semester\Основи програмної інженерії\ЛР_4\2Win\lab 3>
```

Після:

```
PS S:\University\Bachelor\1 year\2 semester\Основи програмної інженерії\ЛР_4\2Win\lab 3> py -m lizard .\src\
=====
NLOC    CCN    token  PARAM  length  location
-----
      3      1     11      0       3 constructor@5-7@.\src\predictionService.js
      1      1      6      0       1 (anonymous)@24-24@.\src\predictionService.js
     23      9    151      3     24 register@14-37@.\src\predictionService.js
      1      1      6      0       1 (anonymous)@45-45@.\src\predictionService.js
      6      4     58      2       6 authenticate@43-48@.\src\predictionService.js
      3      1     10      1       3 validateAge@54-56@.\src\predictionService.js
     13      1     46      5     13 constructor@60-72@.\src\predictionService.js
      3      1     17      0       3 getSummary@74-76@.\src\predictionService.js
      3      1     13      1       3 constructor@80-82@.\src\predictionService.js
     32     16    222      1     37 calculateProbability@91-127@.\src\predictionService.js
      6      1     29      0       6 (anonymous)@140-145@.\src\predictionService.js
      7      4     53      1     13 fetchOdds@134-146@.\src\predictionService.js
1 file analyzed.
=====
NLOC    Avg.NLOC  AvgCCN  Avg.token  function_cnt  file
-----
     108         8.4      3.4      51.8         12  .\src\predictionService.js

=====
!!!! Warnings (cyclomatic_complexity > 15 or length > 1000 or nloc > 1000000 or parameter_count > 100) !!!!
=====
NLOC    CCN    token  PARAM  length  location
-----
     32     16    222      1     37 calculateProbability@91-127@.\src\predictionService.js
=====
Total nloc    Avg.NLOC  AvgCCN  Avg.token  Fun Cnt  Warning cnt  Fun Rt  nloc Rt
-----
     108         8.4      3.4      51.8      12          1    0.08  0.32
PS S:\University\Bachelor\1 year\2 semester\Основи програмної інженерії\ЛР_4\2Win\lab 3>
```

Висновок: Під час виконання лабораторної роботи найціннішим досвідом стало усвідомлення різниці між працюючим кодом і якісним кодом - зокрема, що `Buffer.from(pwd).toString('base64')` не кидає жодної помилки і всі тести проходять, але код містить критичну вразливість безпеки, яку виявив лише Code Review. Робота рецензентом чужого коду розвинула навичку системного читання: замість «чи працює» потрібно думати «чи зрозуміло, чи безпечно, чи підтримувано», що є принципово іншим типом мислення. Практика регресійного тестування після кожного рефакторинг-кроку наочно продемонструвала цінність тестового покриття - без тестів з ЛР 03 було б неможливо впевнено вносити зміни у `authenticate()` при заміні алгоритму хешування. Застосування інструментів статичного аналізу (ESLint з правилом `complexity`, `lizard`) дало змогу кількісно підтвердити покращення коду: метрики до і після рефакторингу - це не суб'єктивна оцінка, а вимірювані факти. Загалом цикл «Code Review → Рефакторинг → Верифікація тестами → Перевірка метрик» сформував чітке розуміння того, як у реальних командах підтримується якість кодової бази без порушення існуючої функціональності.